IE 206: Introduction to AI & Machine Learning

Model Evaluation

Manjesh K. Hanawal

Hypothesis Class

- A hypothesis is a map/function from features ($\mathcal{X}$) to label ($\mathcal{Y}$)

$$h : \mathcal{X} \to \mathcal{Y}$$

- A hypothesis class ($\mathcal{H}$) is the set of such

$$\mathcal{H} = \{h : X \to Y\}$$

- The size of a hypothesis class may be finite or infinite.
- The goal in supervised learning is to identify the best hypothesis from the given class $\mathcal{H}$

Examples of Hypothesis Classes

Linear Functions: represents a straight line in 2D space.

$$h(x) = w_0 + w_1 x \;\; \forall x \in \mathcal{R} \qquad h \equiv (w_0, w_1)$$
$$\mathcal{H} = \{(w_0, w_1) \in \mathcal{R}^2\}$$

Quadratic Functions: represents a parabolic curve.

$$h(x) = w_0 + w_1 x + w_2 x^2 \;\; \forall x \in \mathcal{R} \qquad h \equiv (w_0, w_1, w_2)$$
$$\mathcal{H} = \{(w_0, w_1, w_2) \in \mathcal{R}^3\}$$

Hyperplanes: represents a linear boundary in n-dimensional space.

$$h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b \;\; \forall \mathbf{x} \in \mathcal{R}^d \qquad h \equiv (w_0, w_1, w_2, \ldots, w_d)$$
$$\mathcal{H} = \{(w_0, w_1, w_2, \ldots, w_d) \in \mathcal{R}^{d+1}\}$$

Polynomials: represents polynomial of degree d, covering linear to higher order curves.

Example of hypothesis

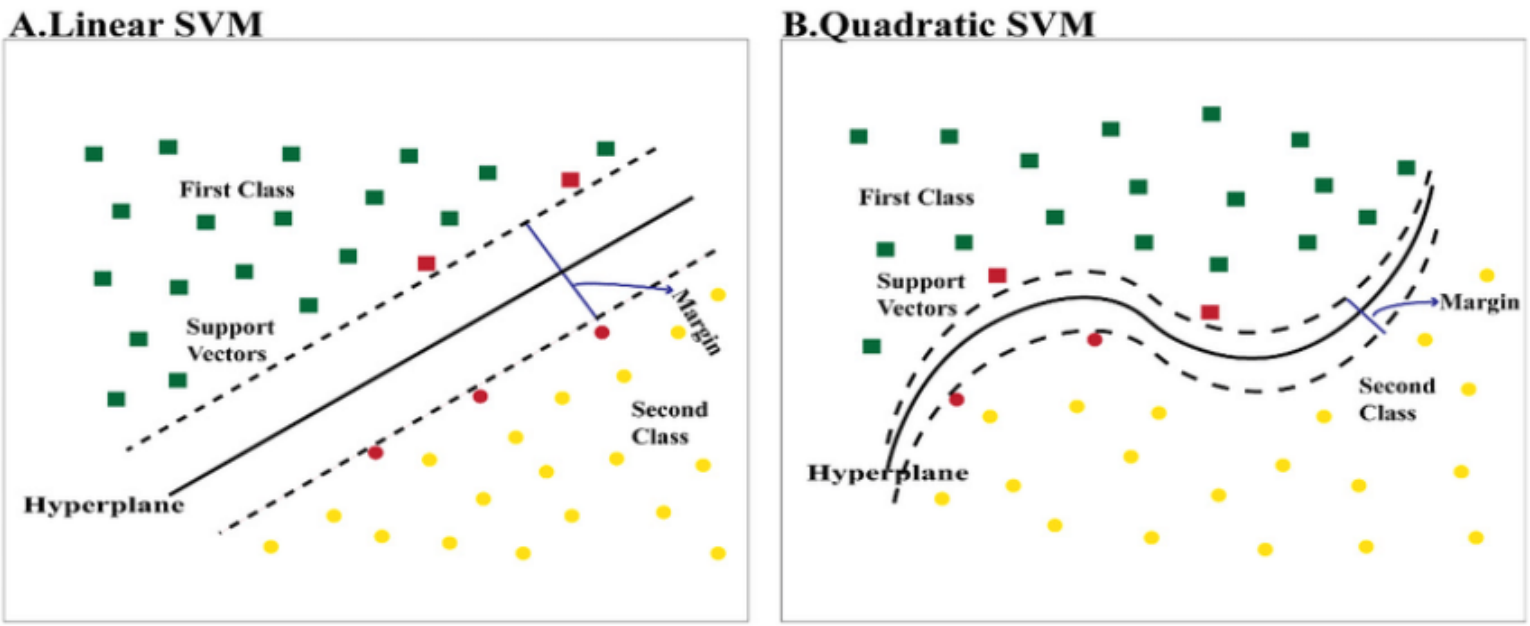

Figure: Representation of Linear and Quadratic hyperplanes (hypothesis classes), which segregate two classes.

Train and Test split

- The train-test split is a technique for evaluating the performance of a machine-learning algorithm.
- **Train Set:** Used to train the model, typically considered 75-80% of the total data.
- **Test Set:** Used to evaluate the models' performance on unseen data, typically 20-30% of the total data.

Notation of Accuracy

- $$Accuracy = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

  Example:
- Correct predictions: 90
- Total predictions: 100
- Accuracy: $90/100 = 0.9$ or 90%

Some definitions

- **True Positive (TP):** Correctly predicted positive instances.
- **True Negative (TN):** Correctly predicted negative instances.
- **False Positive (FP):** Incorrectly predicted positive instances.
- **False Negative (FN):** Incorrectly predicted negative instances.

		Predicted Class: Positive	Predicted Class: Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	**Sensitivity** $\frac{TP}{(TP+FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	**Specificity** $\frac{TN}{(TN+FP)}$
		Precision $\frac{TP}{(TP+FP)}$	**Negative Predictive Value** $\frac{TN}{(TN+FN)}$	**Accuracy** $\frac{TP+TN}{(TP+TN+FP+FN)}$

Confusion Matrix

Figure: Confusion matrix

Examples of TP, TN, FP, FN

Identifying spam emails
Spam is a positive class
Non-spam/legitimate is a negative class

- **True Positive (TP):** a spam email flagged as spam
- **True Negative (TN):** a non-spam email flagged as non-spam
- **False Positive (FP):** a non-spam email flagged as spam
- **False Negative (FN):** a spam email flagged as non-spam (spam got through)

F1 Score, Precision, and Recall

- **Precision:** Measures the proportion of true positive results among all positive predictions.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall:** Measures the proportion of true positive results among all actual positive cases.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

- **F1 Score:** The harmonic mean of precision and recall, providing a single metric that balances both.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1 Score, Precision, and Recall Motivations

- **Precision:** Precision is crucial when the cost of false positives is high (e.g., in spam detection).
- **Recall:** Recall is vital when missing a positive case has a significant cost (e.g., in disease diagnosis).
- **F1 Score:** The F1 score is useful when there is an uneven class distribution or when both precision and recall are equally important. It balances the trade-off between false positives and false negatives.

Underfitting vs Overfitting

Depending on the performance, the model fitting can be divided into three parts:

- Underfitting: High test and training error at low complexity (model too simple) (High Bias and low variance).
- Optimal Complexity: Both training and test errors are low.
- Overfitting: Low training error but high test error at high complexity (model too complex) (Low Bias and high variance).

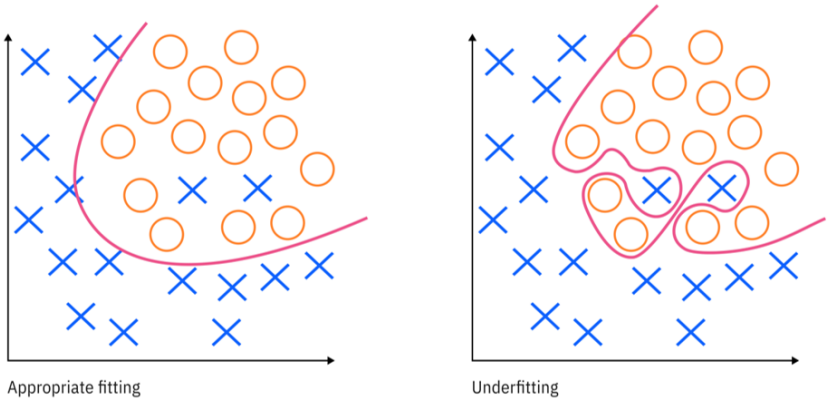

Figure: Model complexity and fitting

Validation set

Why Validation is required?

- **Goal:** Optimize model complexity (for the task) while minimizing under/overfitting.
- We want our model to generalize well without overfitting. This is ensured by validating the model.
- Types of Validation: Hold-out Validation, Cross- validation

Hold-out Validation

- Split data into training and validation sets.

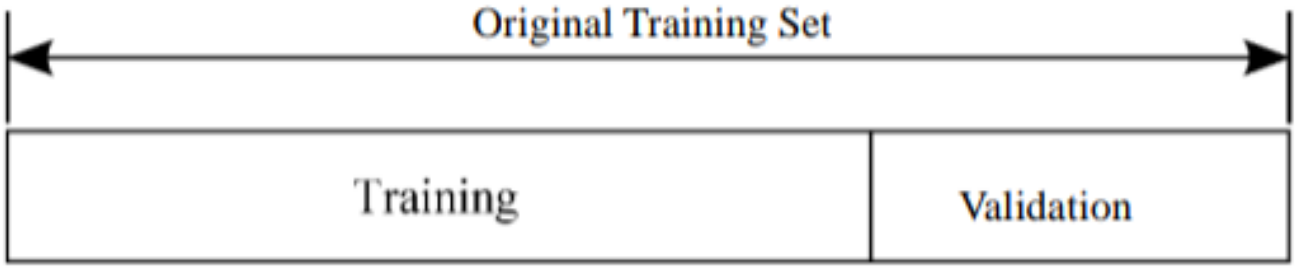

Figure: Hold out validation. Usually set to 20%

- **Cons**
  - Estimation of error rate might be misleading.
  - Wastage of train data.

k-Cross Validation

- Split the data into k subsets or folds.
- For every $i = 1, \cdots, k$
  - train the model on every fold except the ith fold,
  - compute the test error on the ith fold.

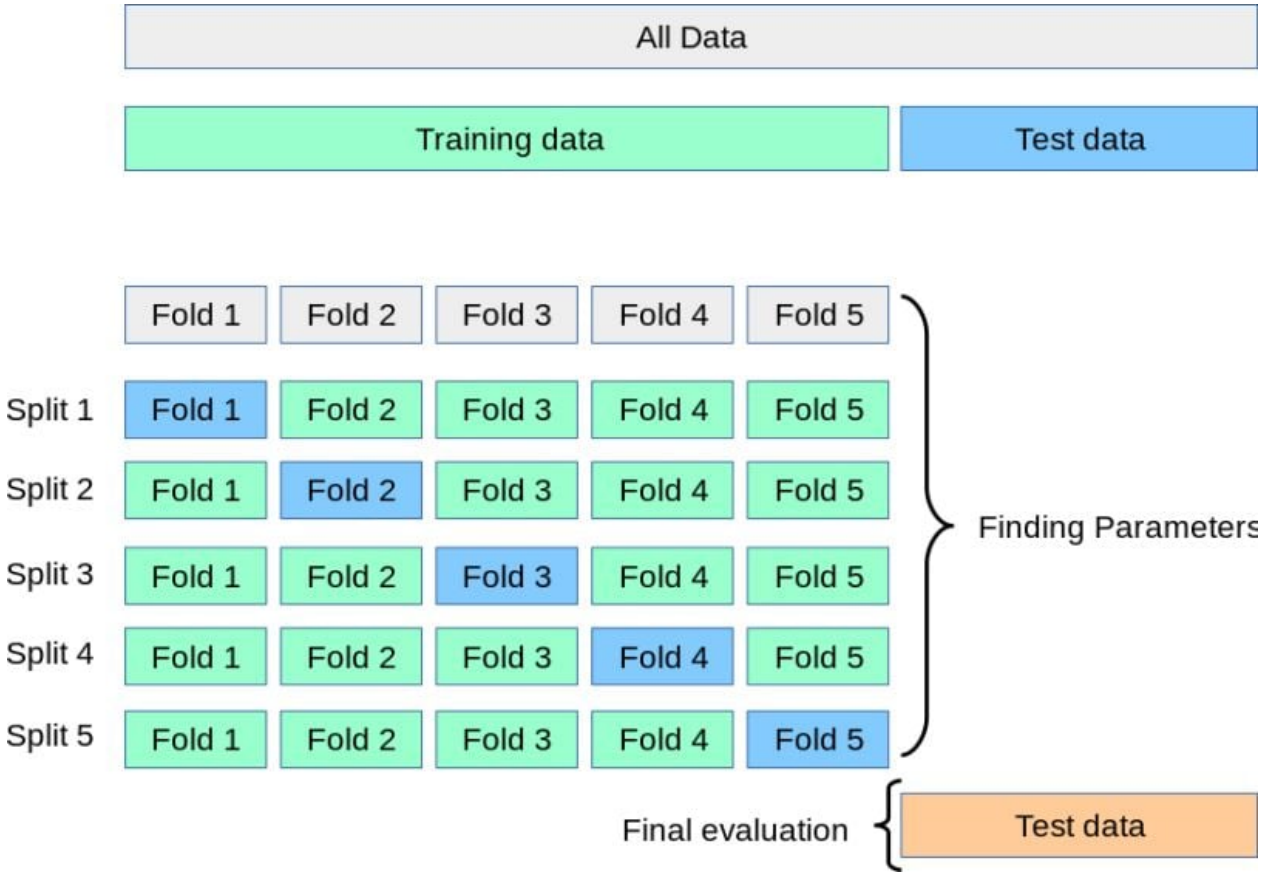

Figure: 5 fold cross validation

Pros and Cons of k-fold cross validation

Pros

- Each sample is used for validation exactly once.
- More tractable than hold-out validation.

Cons

- Can still overfit if we validate too many models!
- Expensive for large k.

Practical Tips

- Usually use K=10
- For larger dataset, choose smaller K

K-Fold Validation

k-Fold Cross Validation for Model Selection

input:

training set $S = (\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)$

set of parameter values Θ

learning algorithm A

integer k

partition S into $S_1, S_2, \ldots, S_k$

foreach $\theta \in \Theta$

for $i = 1 \ldots k$

$h_{i,\theta} = A(S \setminus S_i; \theta)$

$\text{error}(\theta) = \frac{1}{k} \sum_{i=1}^{k} L_{S_i}(h_{i,\theta})$

output

$\theta^\star = \text{argmin}_\theta \left[\text{error}(\theta)\right]$

$h_{\theta^\star} = A(S; \theta^\star)$

Model Selection Curve

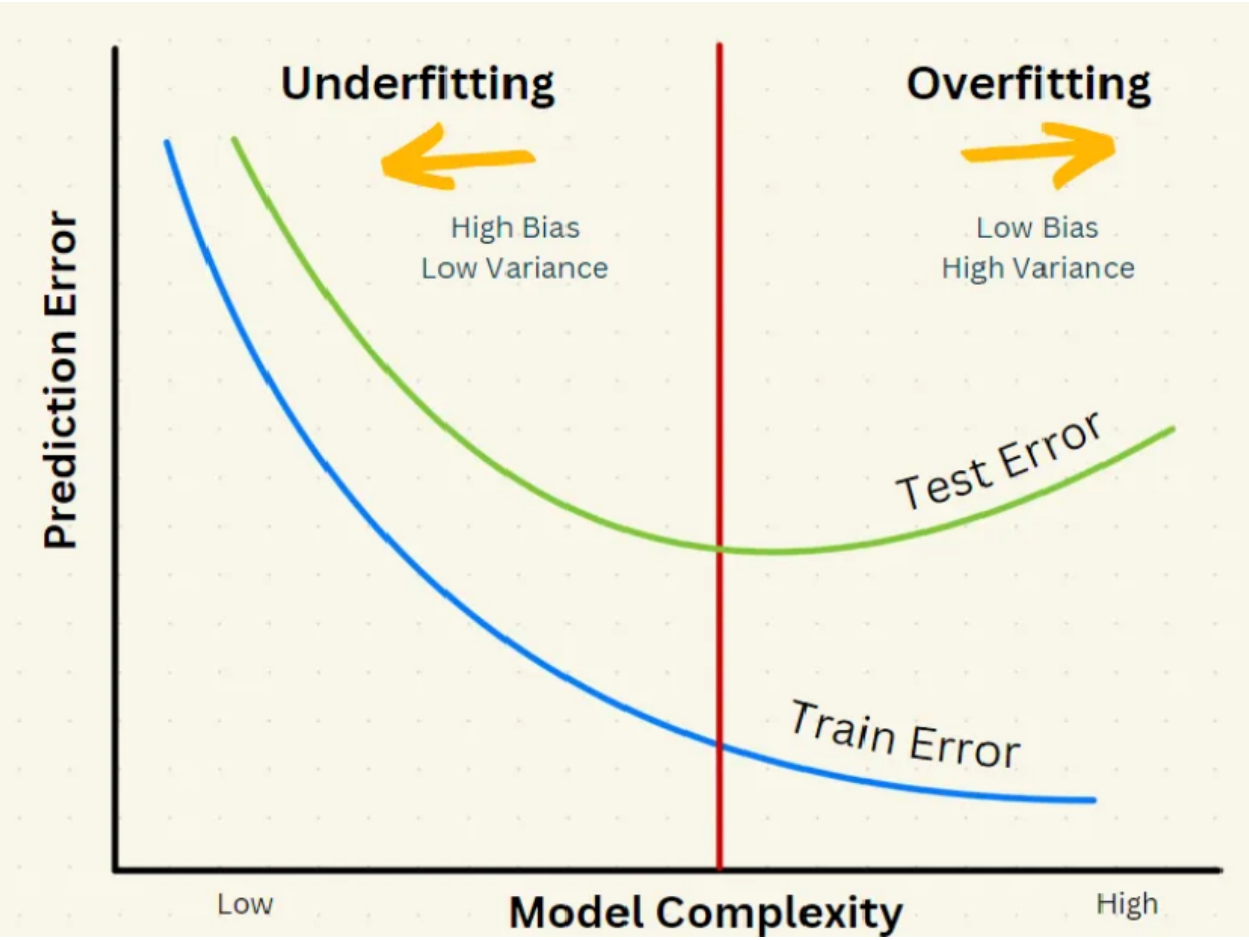

Figure: Train and Test Error vs. Model Complexity

Train and Test Error vs. Complexity

- Training Error: Decreases as model complexity increases. A more complex model can memorize the training data, resulting in lower error.
- Test Error: Initially decreases, but after a certain point, begins to increase. More complexity leads to a model that performs poorly on new unseen data due to memorization.

Practical implications

- Select a model with sufficient complexity to capture the underlying data patterns but not so complex that it overfits and performs poorly on new data.
- The main issue is that it is difficult to predict the region where overfitting starts.
- Solution: Regularization and cross-validation!